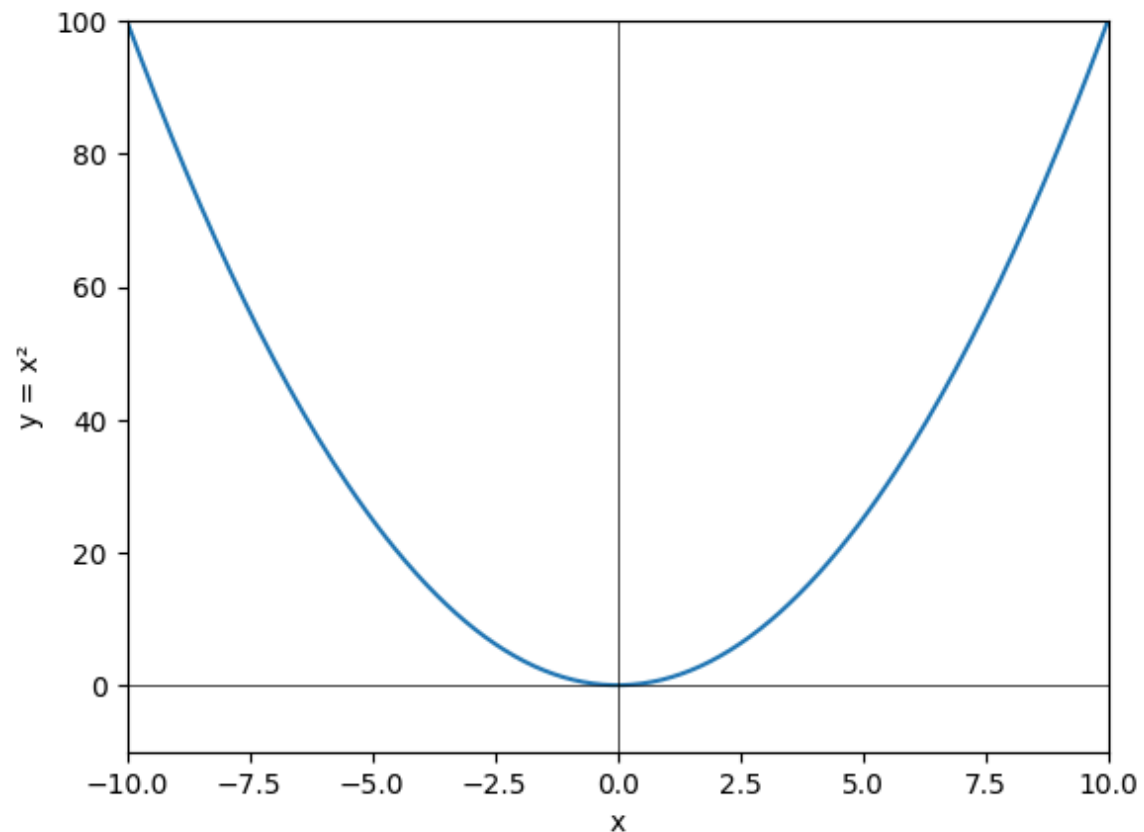


Funció quadràtica

```
In [5]: plot_quadratic(xs, ylim=(-10, 100), ylabel='y = x2'); plt.show()
```



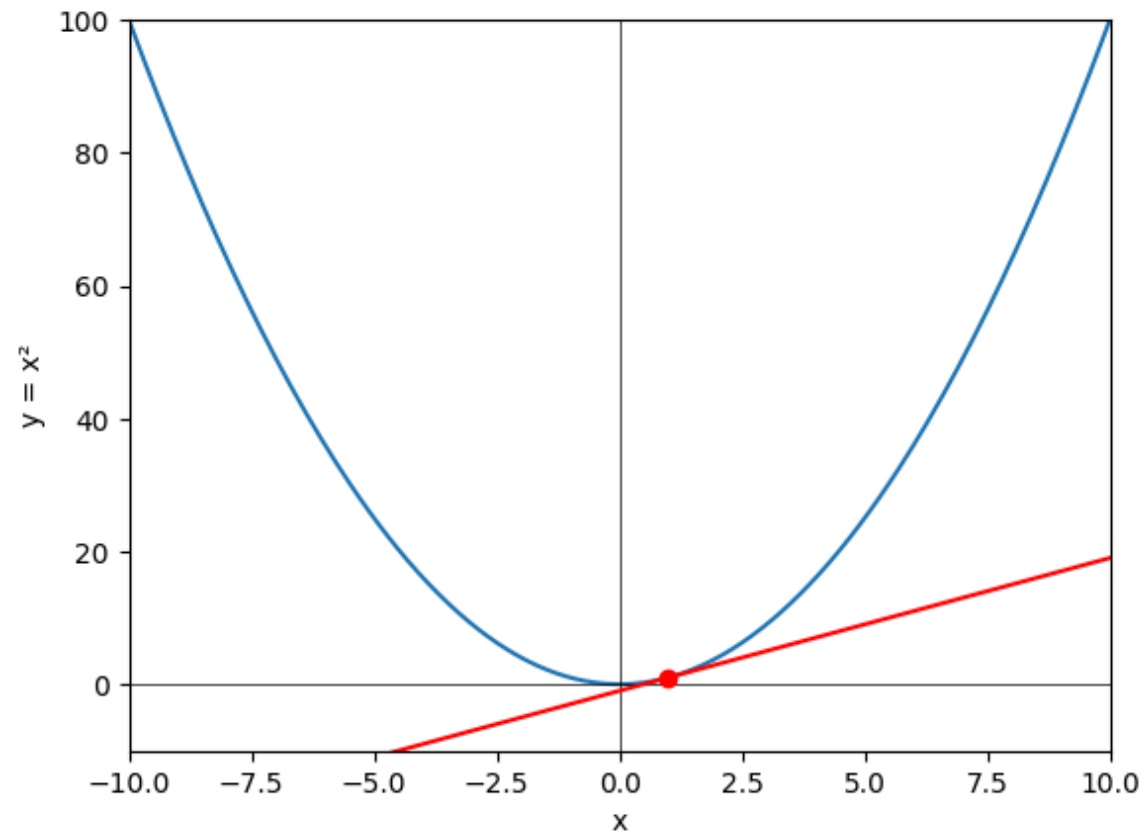
Derivada de la funció quadràtica

- Quadràtica: $f(x) = x^2$
- Derivada: $f'(x) = 2x$
 - Pendent de la recta tangent en un punt donat de $f(x)$.
- Pendent: canvi en $f(x)$ per cada unitat de x .
 - Com més gran és el pendent, més gran és la taxa de canvi.
 - Els pendents negatius indiquen una tendència decreixent.
 - Els pendents positius indiquen una tendència creixent.
 - Un pendent de 0 significa que no hi ha canvi. P.ex., el mínim en les funcions quadràtiques.

```
In [9]: plot_with_slope(x=1)  
plt.show()
```

Point: (1, 1)

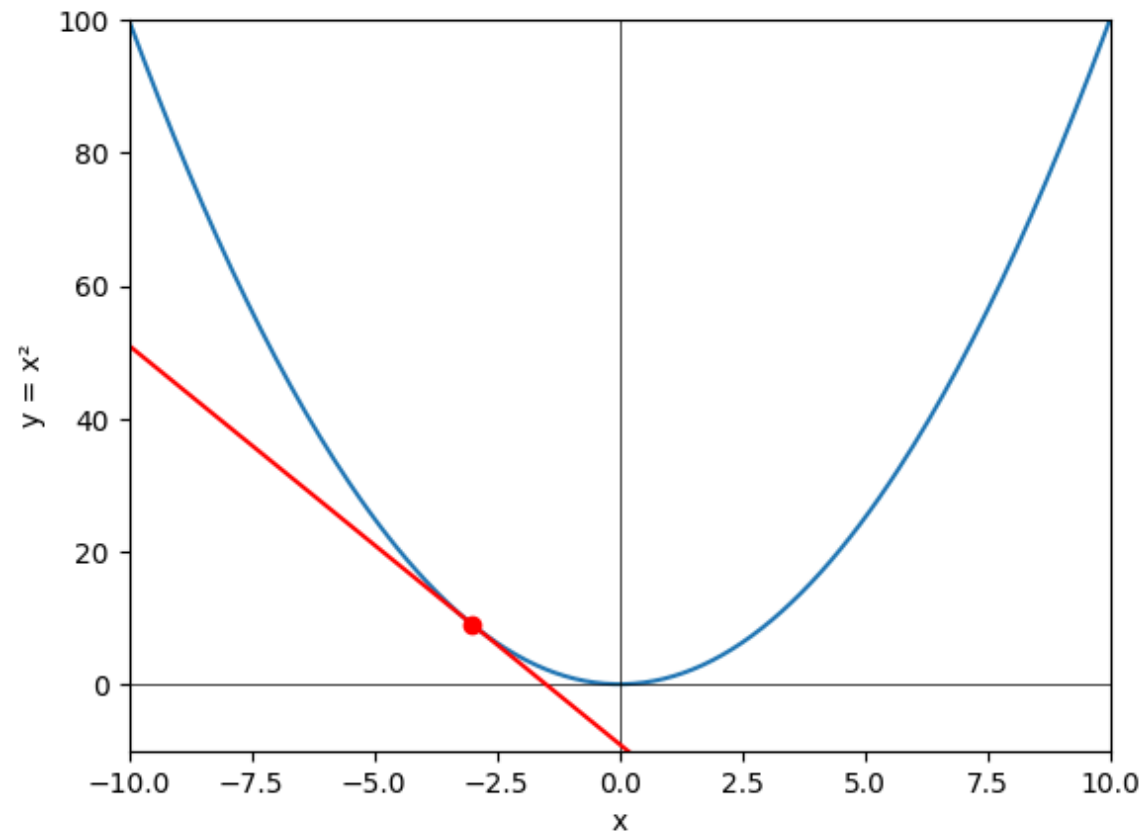
Slope: 2



```
In [10]: plot_with_slope(x=-3)  
plt.show()
```

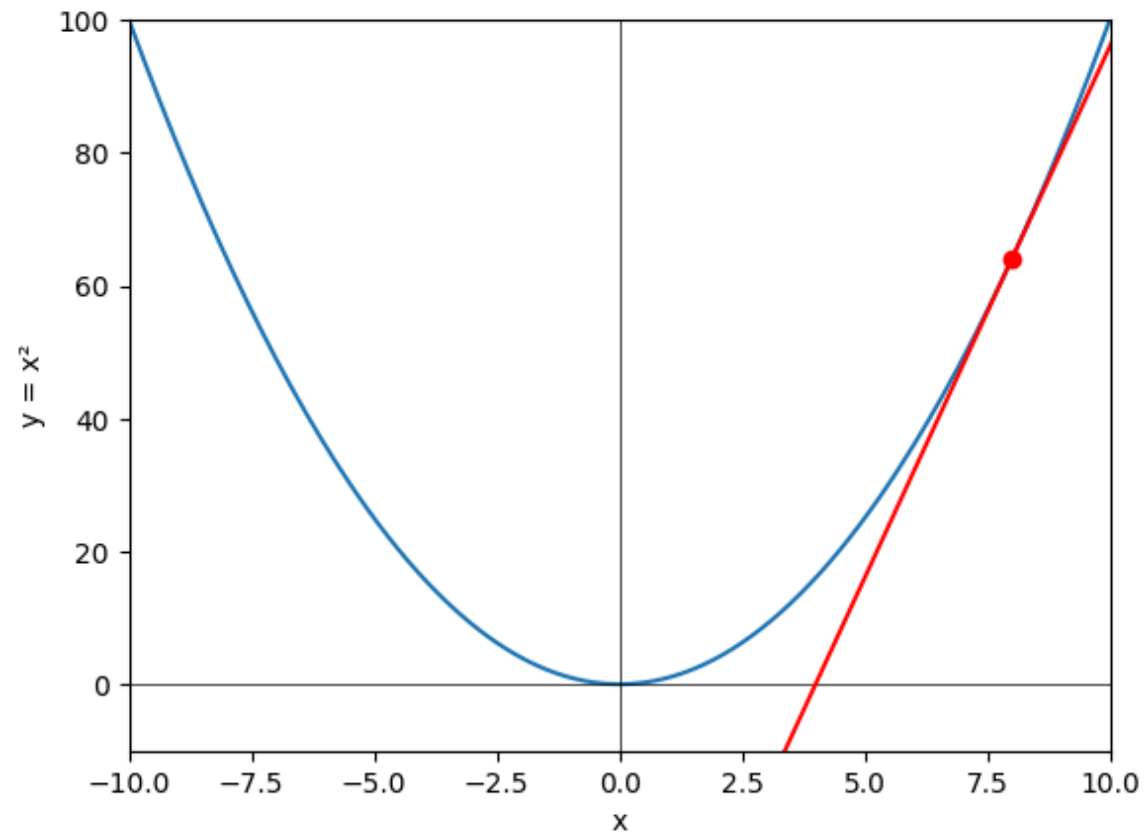
Point: (-3, 9)

Slope: -6



```
In [11]: plot_with_slope(x=8)
plt.show()
```

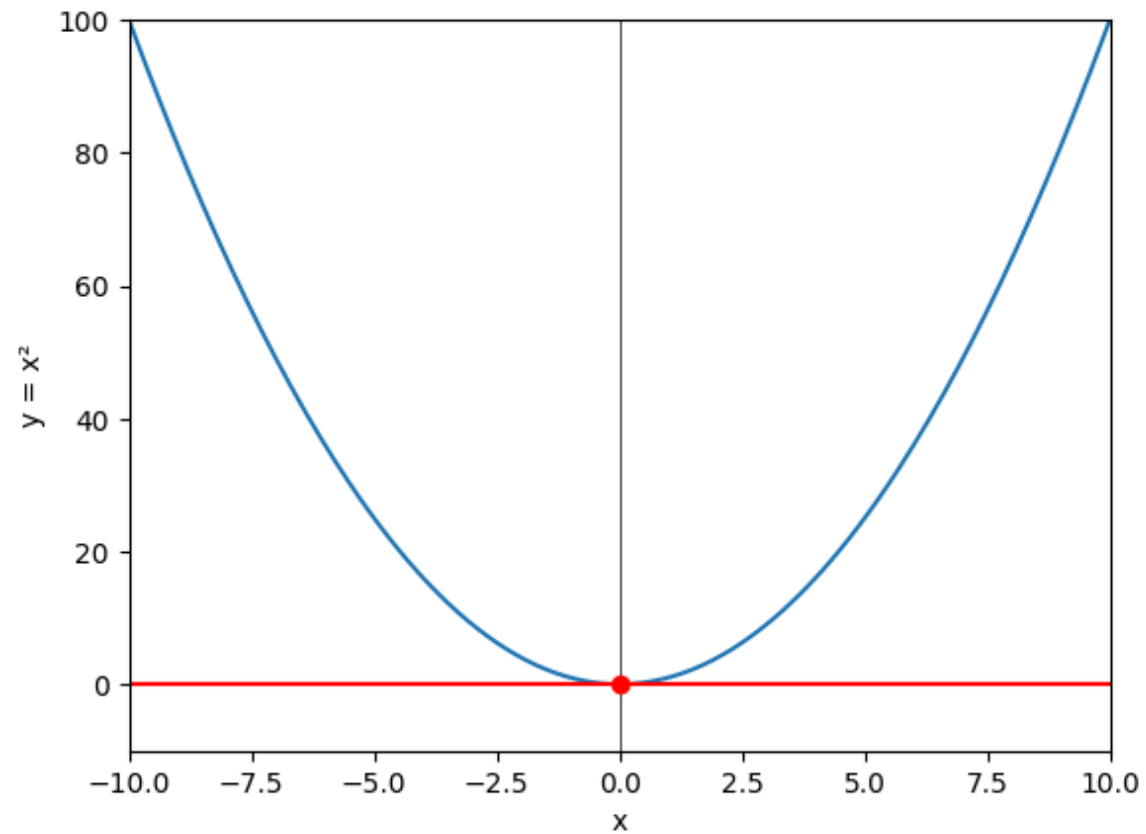
Point: (8, 64)
Slope: 16



```
In [12]: plot_with_slope(x=0)
plt.show()
```

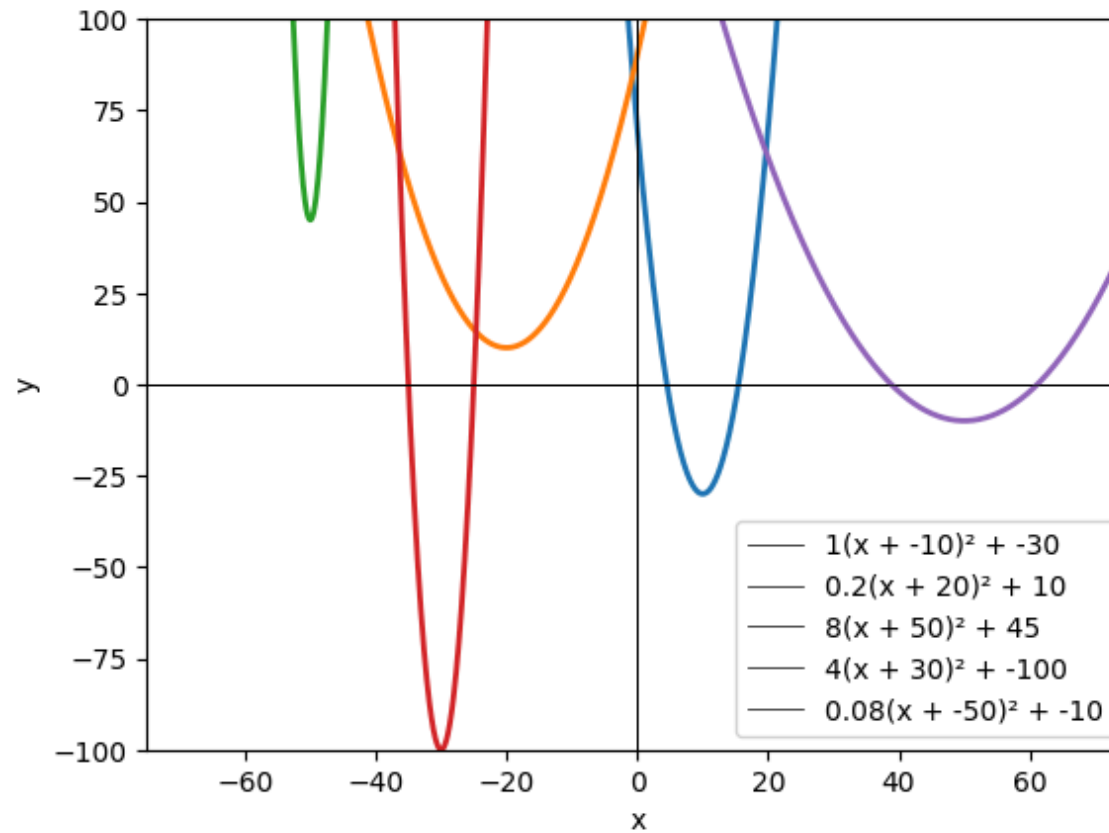
Point: (0, 0)

Slope: 0



- Les fonctions quadràtiques poden tenir moltes formes i posicions diferents.

In [14]: `plot_quadratics()`

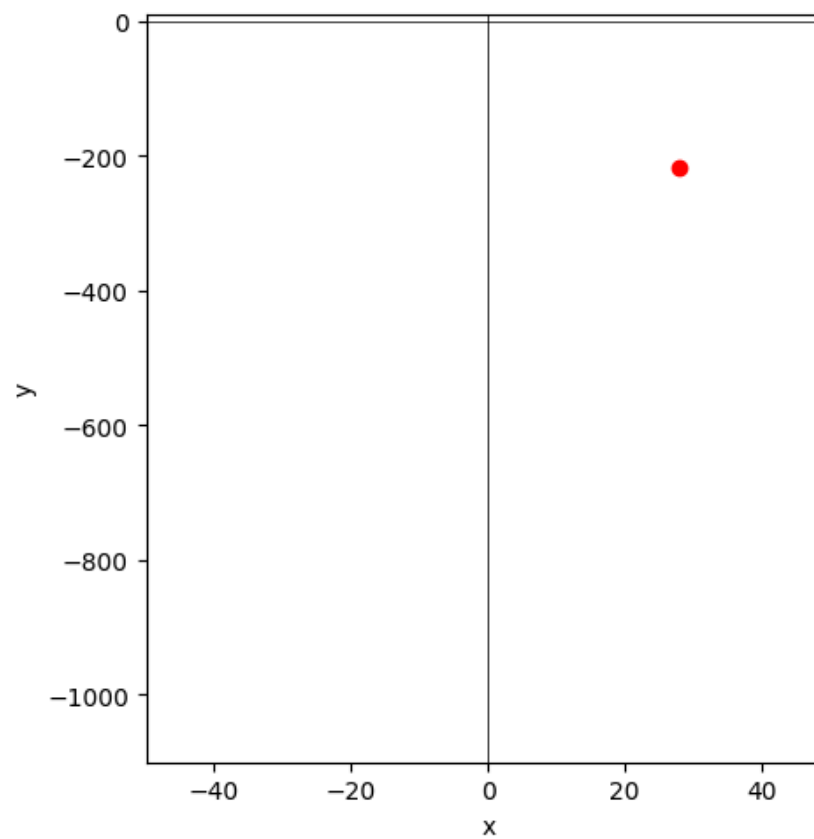


Gradient Descent

- És fàcil trobar el mínim si dibuixem una funció quadràtica.
- També podem calcular el mínim amb una mica de càlcul i àlgebra.
- Però què passaria si dibuixar o calcular el mínim fos costós computacionalment?
- Com podem trobar on és el mínim global?

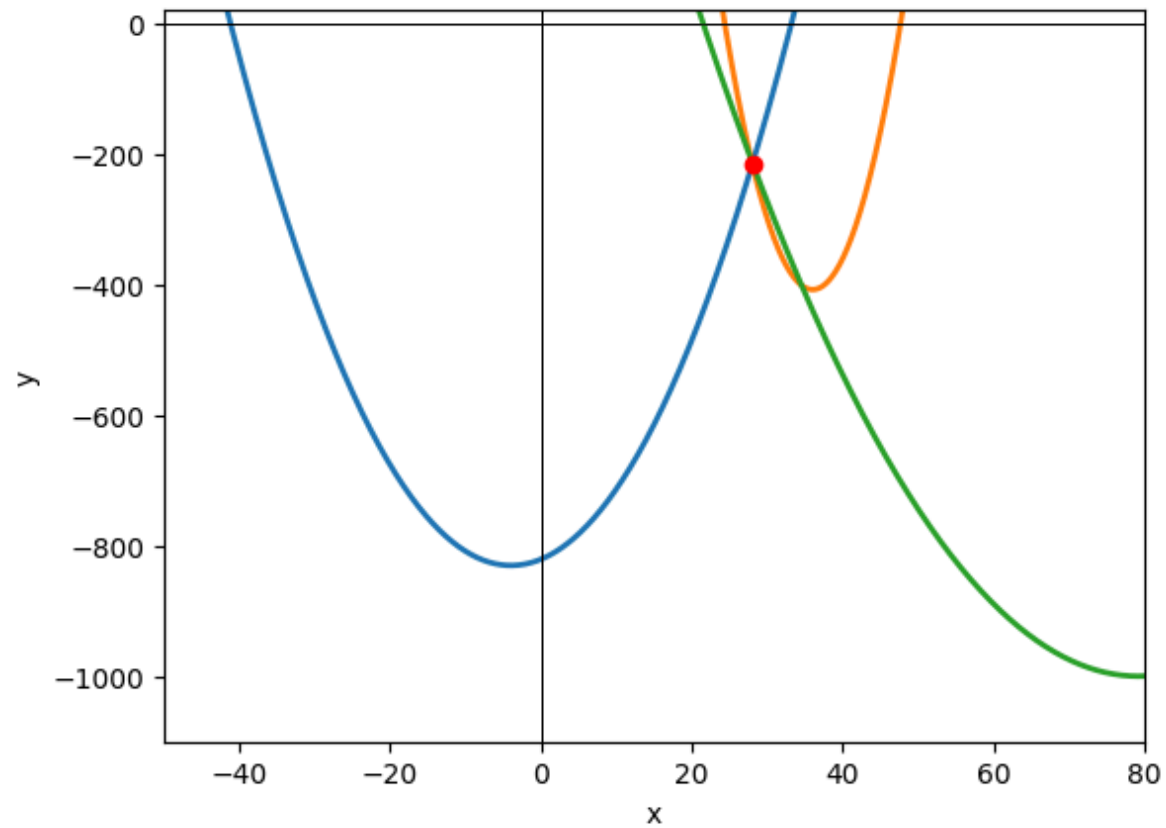

```
In [16]: plot_hidden_quadratic(x=28)
```

Initial Point: (28, -216)



- A priori no coneixem la forma de la paràbola.
- Podria ser qualsevol.

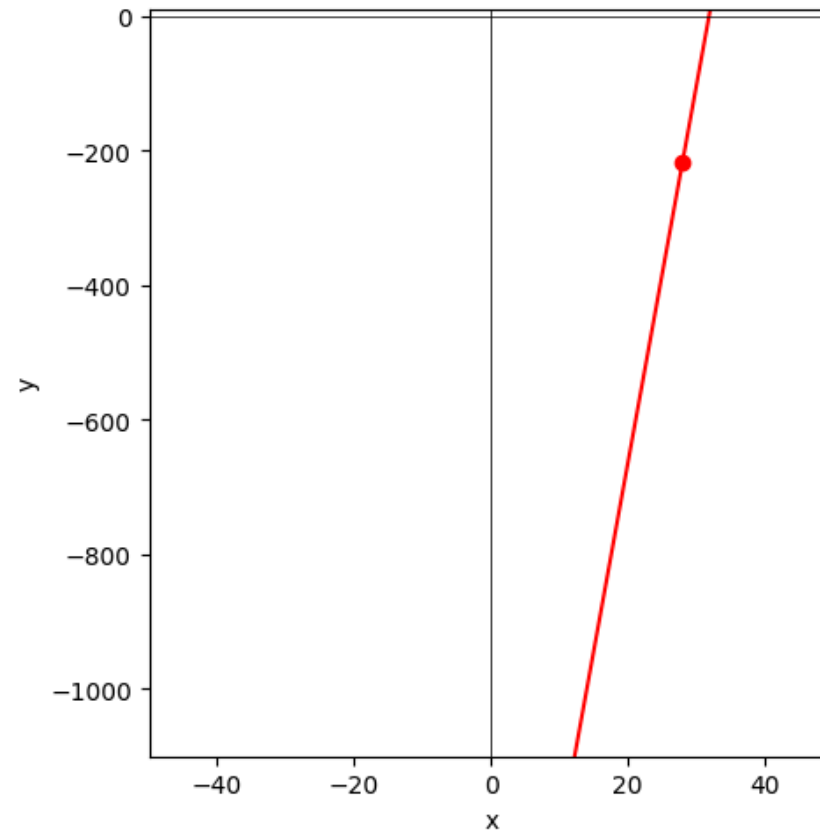
```
In [18]: plot_quad_crossings()
```



- Podem calcular el pendent, això és fàcil.
- Ara sabem que estem lluny del mínim i a la part dreta de la paràbola.

```
In [19]: plot_hidden_quadratic(x=28, show_slope=True); plt.show()
```

Initial Point: (28, -216)
Initial Slope: 56



- Una opció per trobar el mínim seria aplicar el gradient descent.
- Consisteix a iterar i actualitzar el valor de x .
- $x_{\text{new}} = x_{\text{old}} - \alpha \cdot \nabla_x f(x)$, on
 - x_{old} és el valor actual de x .
 - x_{new} és el nou valor de x després de l'actualització.
 - α és el learning rate. Defineix amb quina rapidesa actualitzem x .
 - $\nabla_x f(x)$ és el gradient de la funció. És el pendent en el nostre exemple.

```
In [20]: plot_hidden_quadratic(x=28, show_slope=True, iters=1,  
                                learning_rate=0.1, show_final=False)  
plt.show()
```

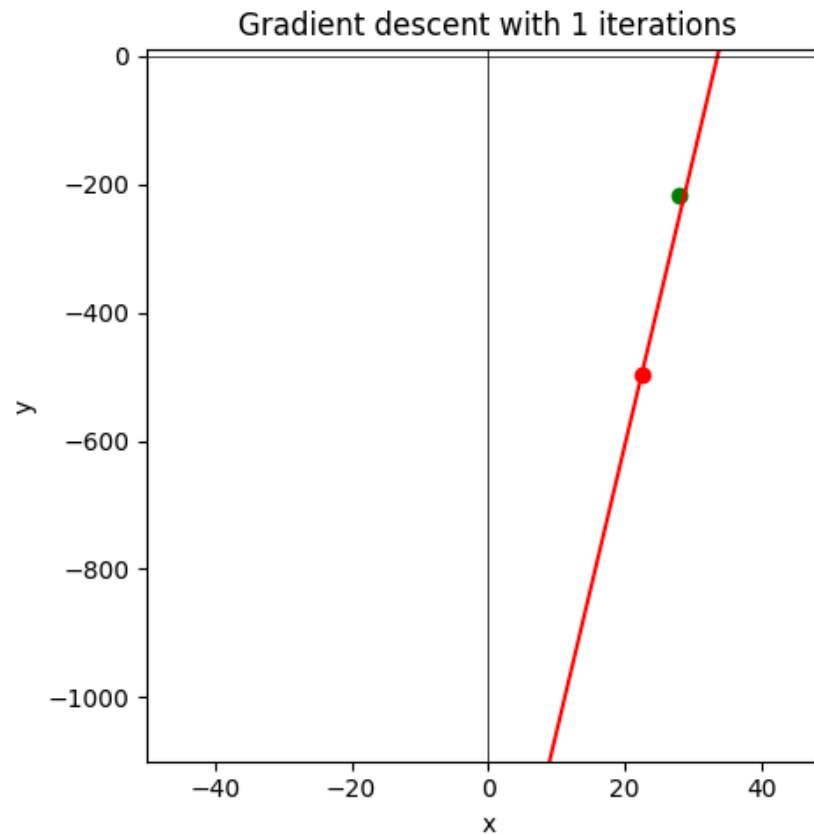
Initial Point: (28, -216)

Initial Slope: 56

Iteration 1: $x_{new} = 28 - 0.1 * 56 = 22.4$

New point: (x = 22.4, y = -498.2)

New slope: 44.8

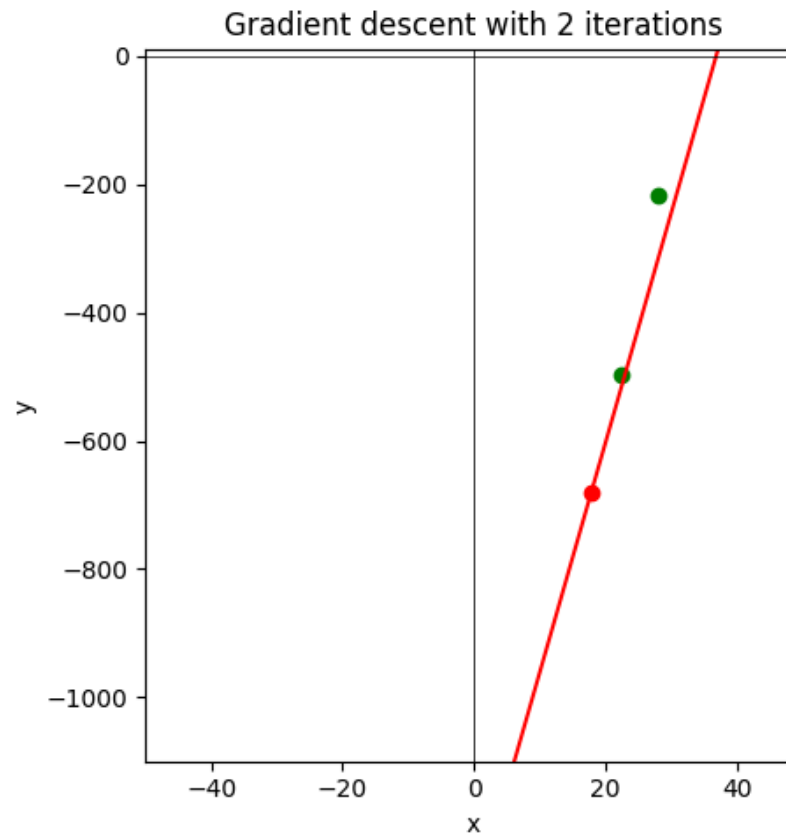


```
In [21]: plot_hidden_quadratic(x=28, show_slope=True, iters=2,  
                                learning_rate=0.1, show_final=False)  
plt.show()
```

Initial Point: (28, -216)
Initial Slope: 56

Iteration 1: $x_{new} = 28 - 0.1 * 56 = 22.4$
New point: (x = 22.4, y = -498.2)
New slope: 44.8

Iteration 2: $x_{new} = 22.4 - 0.1 * 44.8 = 17.9$
New point: (x = 17.9, y = -679.6)
New slope: 35.8



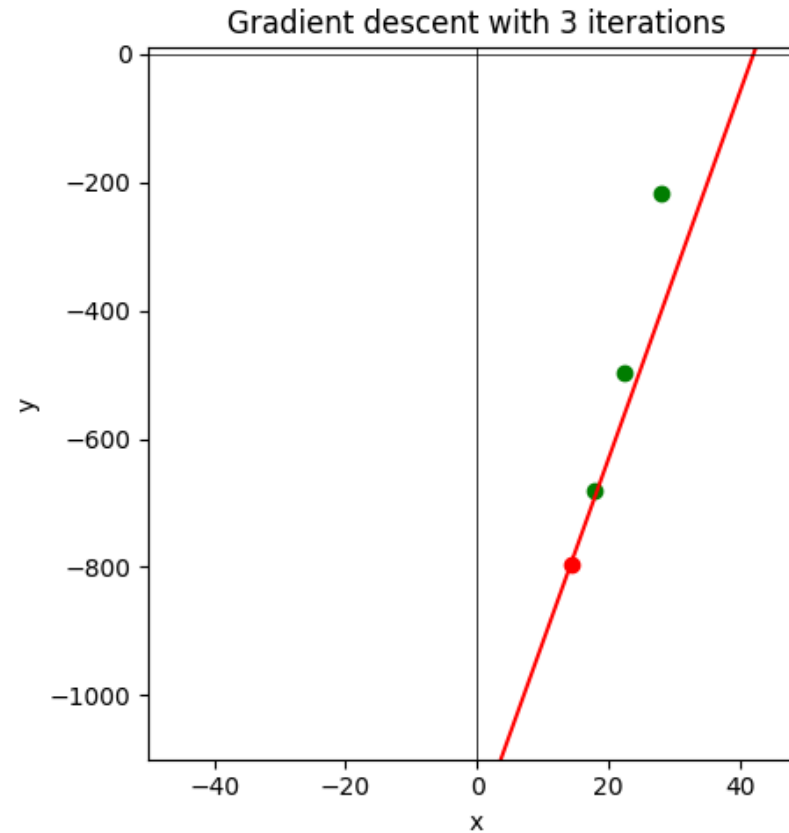
```
In [22]: plot_hidden_quadratic(x=28, show_slope=True, iters=3,  
                                learning_rate=0.1, show_final=False)  
plt.show()
```

Initial Point: (28, -216)
Initial Slope: 56

Iteration 1: $x_{new} = 28 - 0.1 * 56 = 22.4$
New point: (x = 22.4, y = -498.2)
New slope: 44.8

Iteration 2: $x_{new} = 22.4 - 0.1 * 44.8 = 17.9$
New point: (x = 17.9, y = -679.6)
New slope: 35.8

Iteration 3: $x_{new} = 17.9 - 0.1 * 35.8 = 14.3$
New point: (x = 14.3, y = -795.5)
New slope: 28.6



```
In [23]: plot_hidden_quadratic(x=28, show_slope=True, iters=4,  
                                learning_rate=0.1, show_final=False)  
plt.show()
```

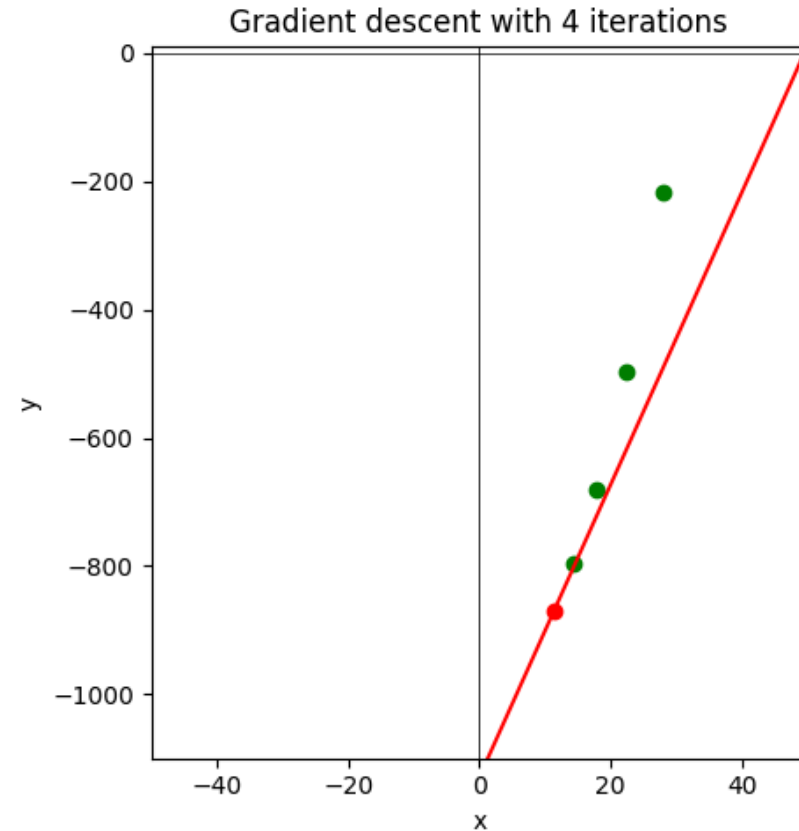
Initial Point: (28, -216)
Initial Slope: 56

Iteration 1: $x_{new} = 28 - 0.1 * 56 = 22.4$
New point: (x = 22.4, y = -498.2)
New slope: 44.8

Iteration 2: $x_{new} = 22.4 - 0.1 * 44.8 = 17.9$
New point: (x = 17.9, y = -679.6)
New slope: 35.8

Iteration 3: $x_{new} = 17.9 - 0.1 * 35.8 = 14.3$
New point: (x = 14.3, y = -795.5)
New slope: 28.6

Iteration 4: $x_{new} = 14.3 - 0.1 * 28.6 = 11.4$
New point: (x = 11.4, y = -870.0)
New slope: 22.8

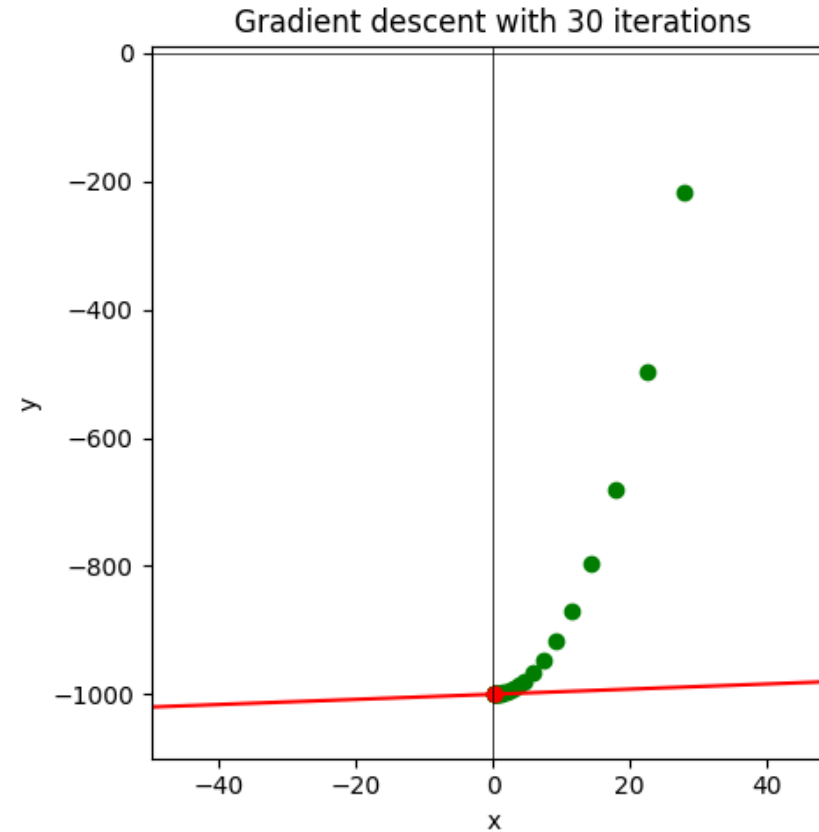



```
In [24]: plot_hidden_quadratic(x=28, show_slope=True, iters=30,  
                                learning_rate=0.1, verbose=False); plt.show()
```

Initial Point: (28, -216)
Initial Slope: 56

...

Final Point: (0.2, -1000.0)
Final Slope: 0.4



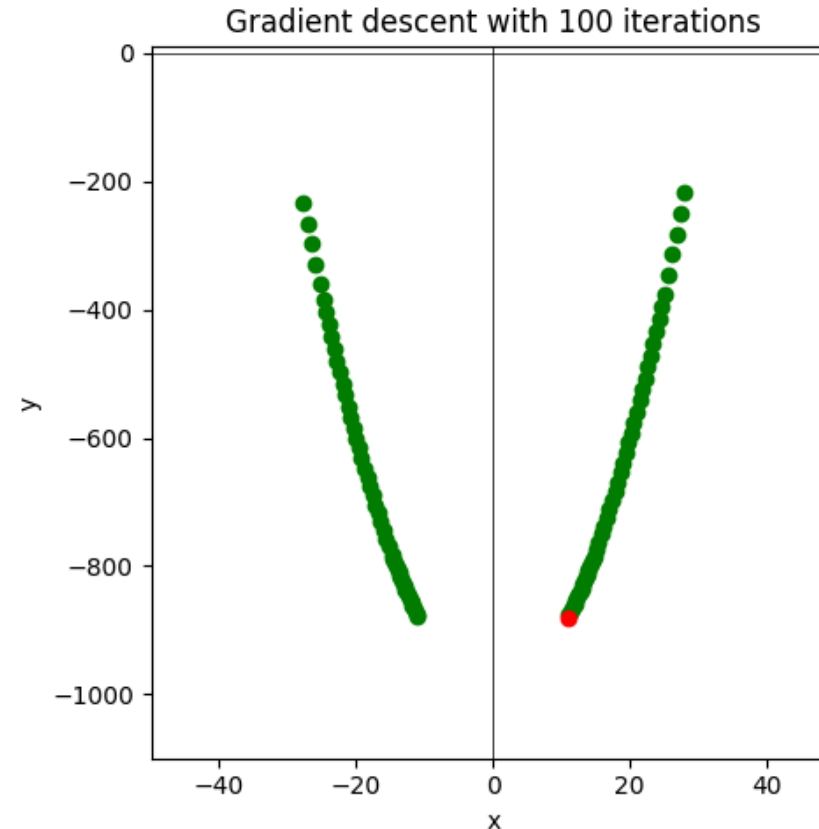
- Compte amb l'elecció del learning rate!
- Els valors grans rebotaran entre els "dos pendents" de la funció quadràtica:
 - Pot trigar més temps a convergir.
 - O pot no convergir mai.

```
In [25]: plot_hidden_quadratic(x=28, show_slope=False, iters=100,  
                                learning_rate=0.995, verbose=False); plt.show()
```

Initial Point: (28, -216)

...

Final Point: (10.9, -881.2)



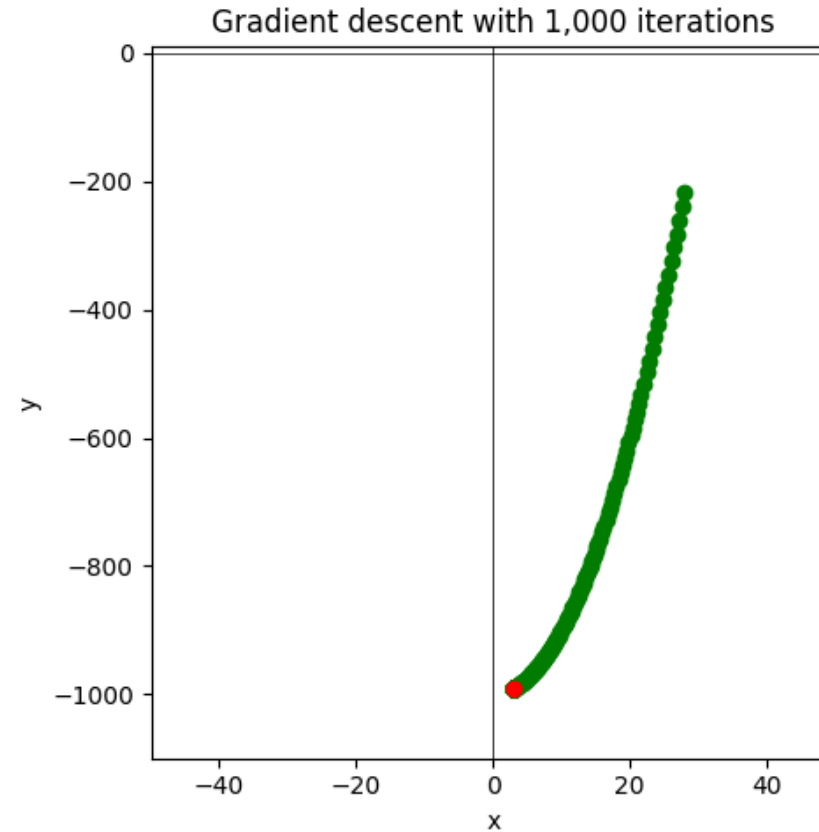
- Els learning rates baixos trigaran més a convergir.
 - Tot i que finalment arribaran a algun punt.
 - O no si la funció té més d'un mínim.

```
In [26]: plot_hidden_quadratic(x=28, show_slope=False, iters=1_000,  
                                learning_rate=0.008, verbose=False); plt.show()
```

Initial Point: (28, -216)

...

Final Point: (3.1, -990.4)



Linear Regression

- Suposem que tenim una sola feature x .
- Predicció de la Linear Regression:
 - $\hat{y}_i = w_0 + w_1 \cdot x_i$
- Hem dit que la LR optimitza el MSE .
- $MSE = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$
- El MSE és una funció quadràtica respecte als coeficients w_n .
 - Podem aplicar GD!
- $MSE = \frac{1}{N} \sum (y_i - w_0 + w_1 \cdot x_i)^2$

```
In [28]: df = pd.DataFrame({
          'm2': [50, 80, 100, 120, 150, 200],
          'price': [60000, 96000, 100000, 130000, 140000, 199000]
        })
df
```

```
Out[28]:
```

	m2	price
0	50	60000
1	80	96000
2	100	100000
3	120	130000
4	150	140000
5	200	199000

```
In [29]: # m2
x = df['m2'].values

# euros
y = df['price'].values
```

- La Linear Regression troba els coeficients w_i que optimitzen el MSE :
 - $\hat{y}_i = w_0 + w_1 \cdot x_i$
- Suposem, per a aquest exemple, que sabem que el w_0 òptim és 0.
- Podem trobar el valor de w_1 que dóna el millor MSE ?
- Sí! Per exemple, aplicant el gradient descent!
- Podem començar amb un w_1 aleatori i descendir fins a la convergència.

In [34]: `w1 = -3894`

- Sabem que $w_0 = 0$.
- I hem fixat un $w_1 = -3894$ aleatori.
- Ara podem calcular totes les prediccions $\hat{y}_i = w_0 + w_1 \cdot x_i$.
- A partir dels x_i (metres quadrats de l'habitatge) del nostre dataframe.
- Com que podem calcular les prediccions $\hat{y}$, podem calcular el MSE .
 - $MSE = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$

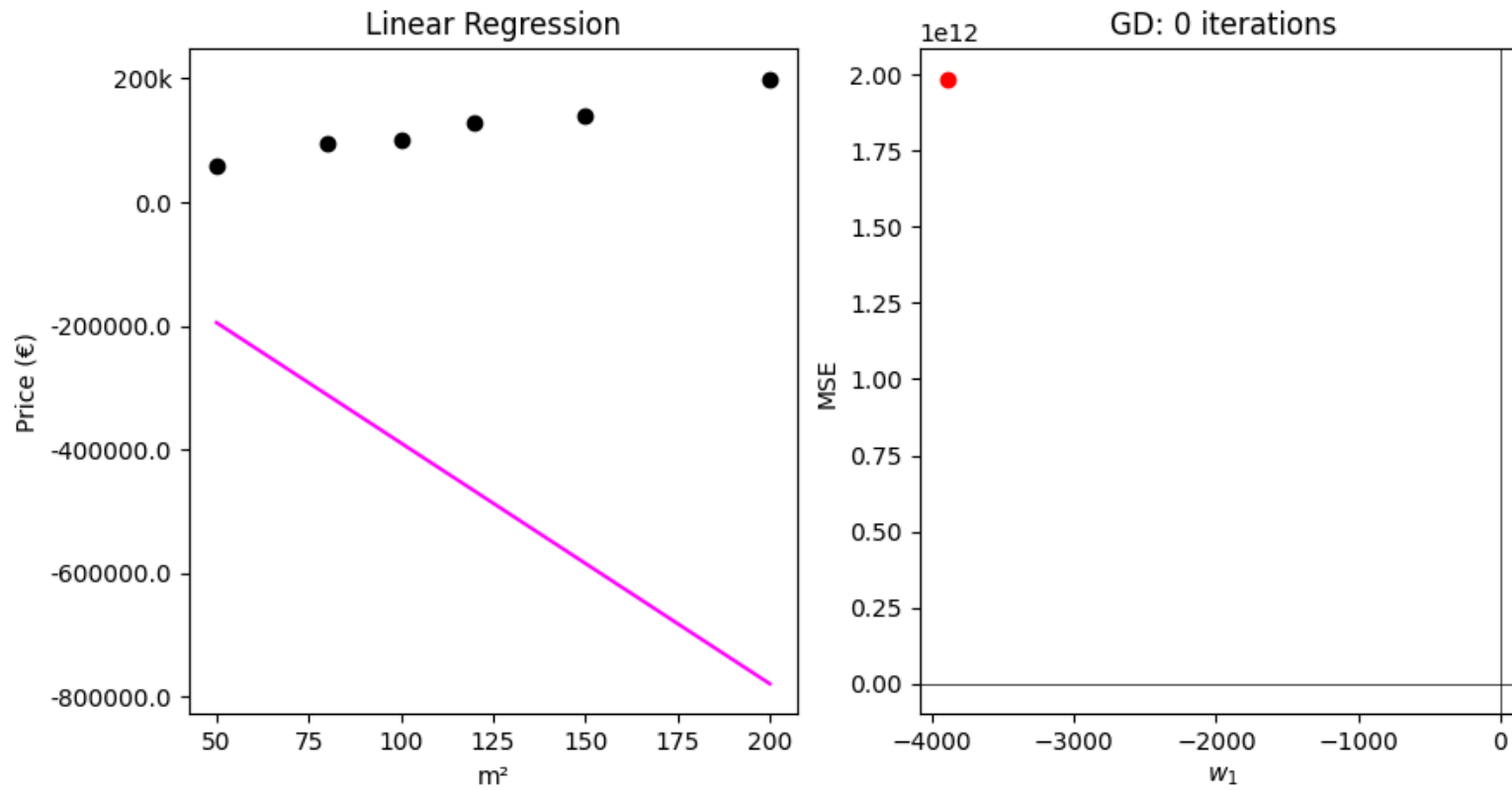
- Tenim una mala predicció (magenta) perquè hem triat w_1 aleatòriament.
- El nostre MSE (punt vermell) és altíssim.
- No coneixem la forma del nostre MSE quadràtic,
- Però podem calcular el pendent.
- I aplicar el gradient descent:

$$\blacksquare w_1 = w_1 - \alpha \cdot \nabla_{w_1} MSE$$

```
In [36]: w1 = plot_lr_gd(w1, x, y, iters=0)
```

w1: -3894.0

Slope: -156,708,400

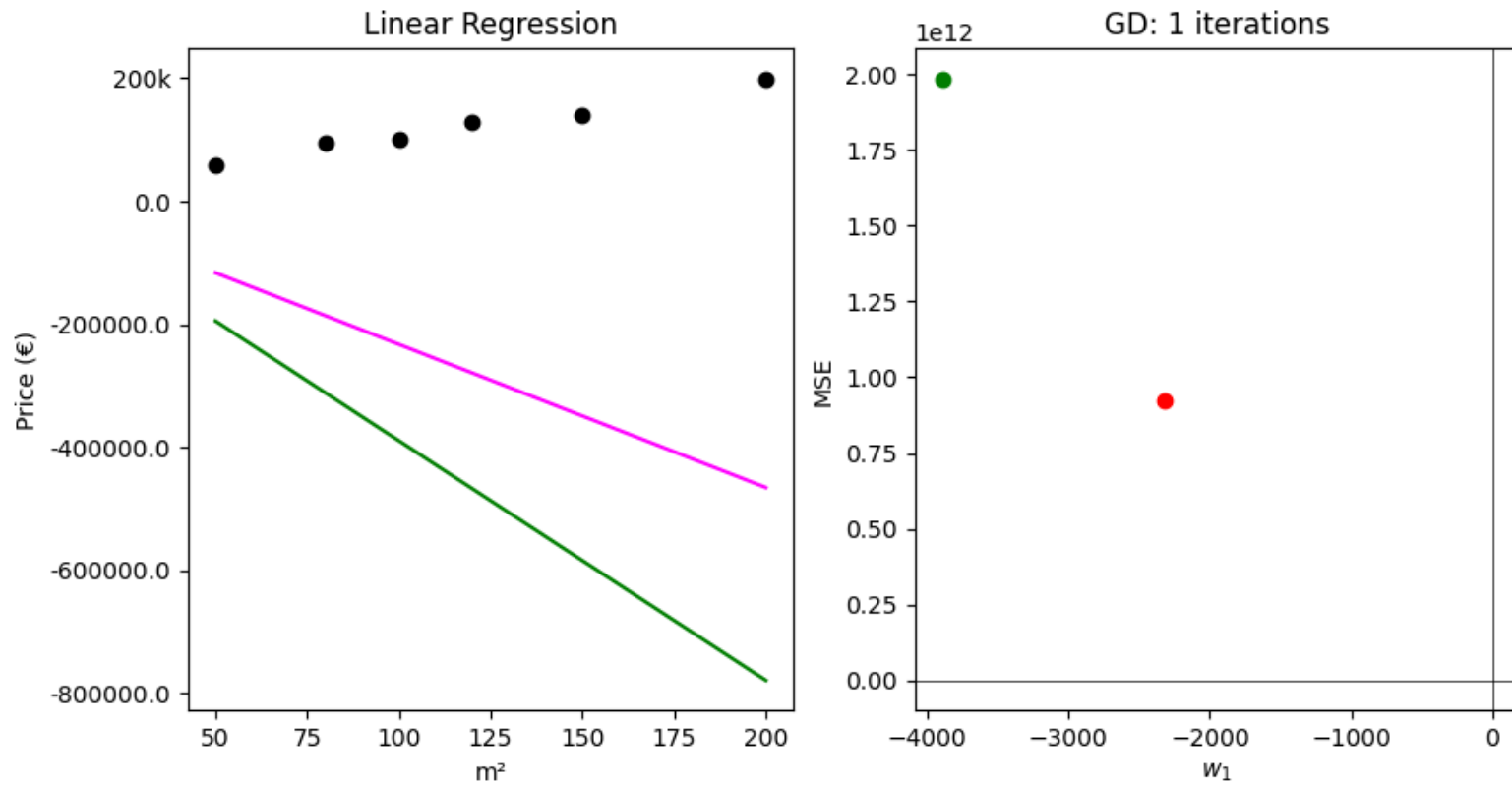


```
In [37]: learning_rate = 0.00001
```

```
In [38]: w1 = plot_lr_gd(w1, x, y, iters=1)
```

New w1: -2326.9

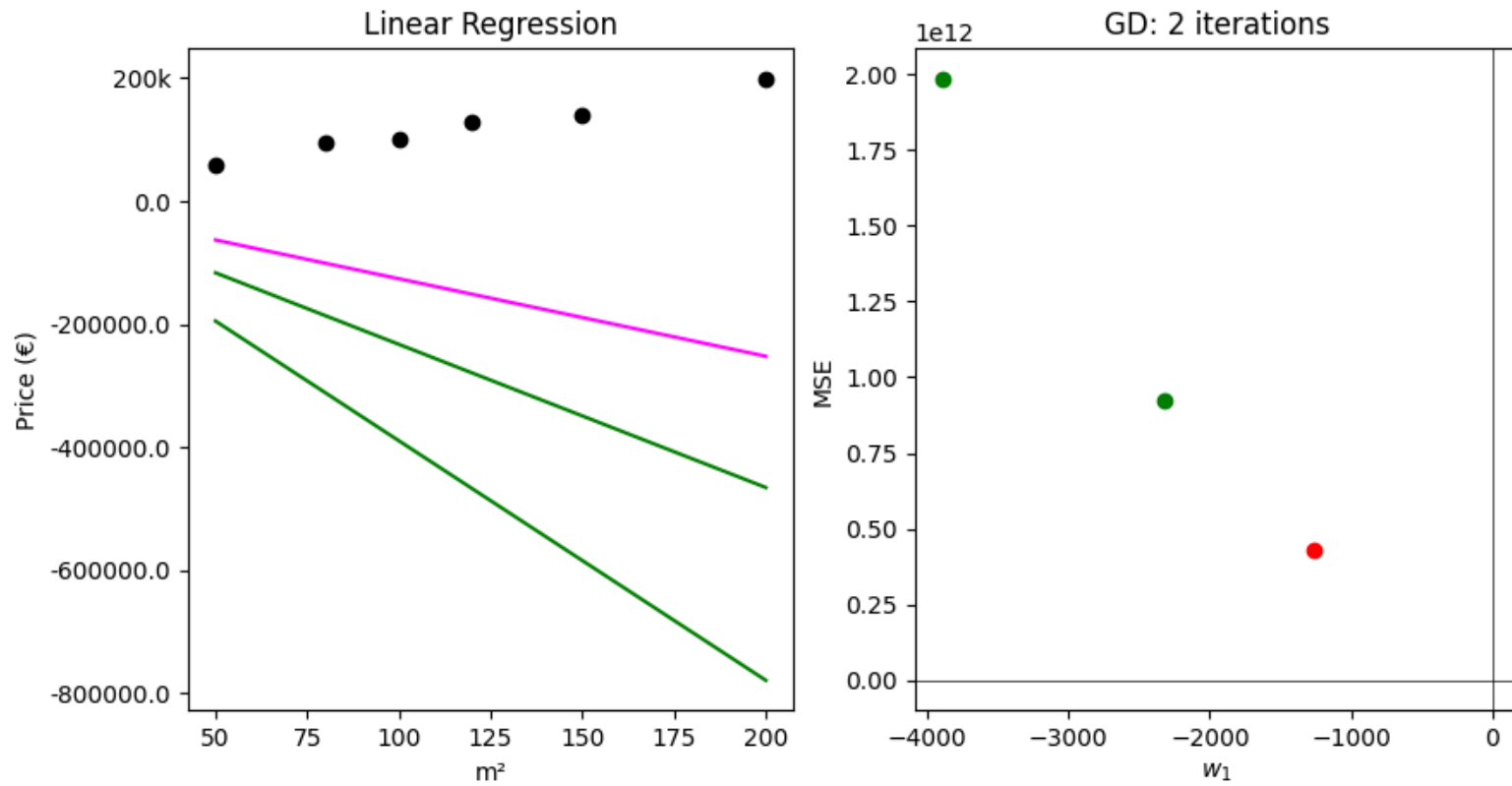
Slope: -106,666,184



```
In [39]: w1 = plot_lr_gd(w1, x, y, iters=2)
```

New w1: -1260.3

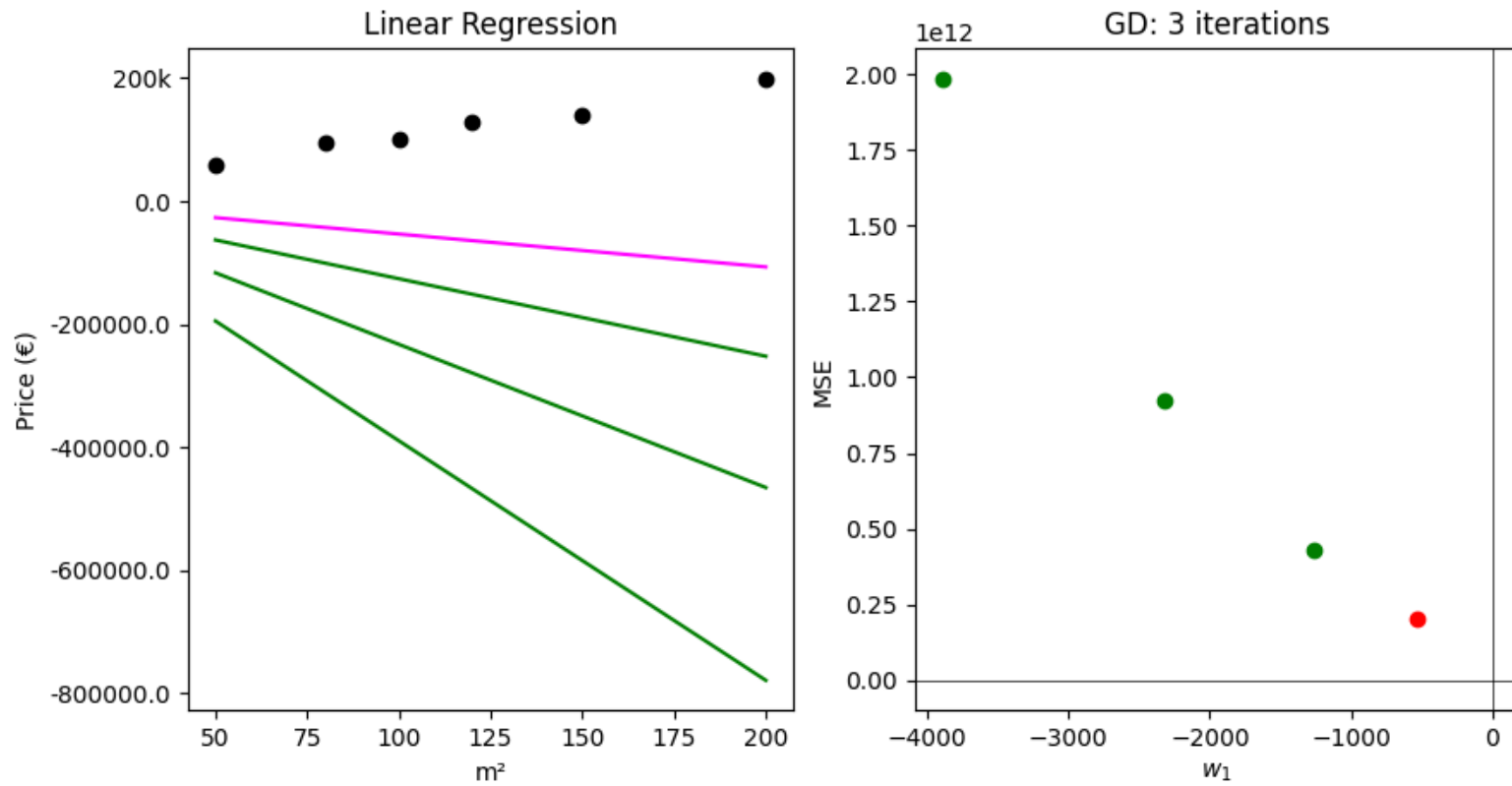
Slope: -72,604,116



```
In [40]: w1 = plot_lr_gd(w1, x, y, iters=3)
```

New w1: -534.2

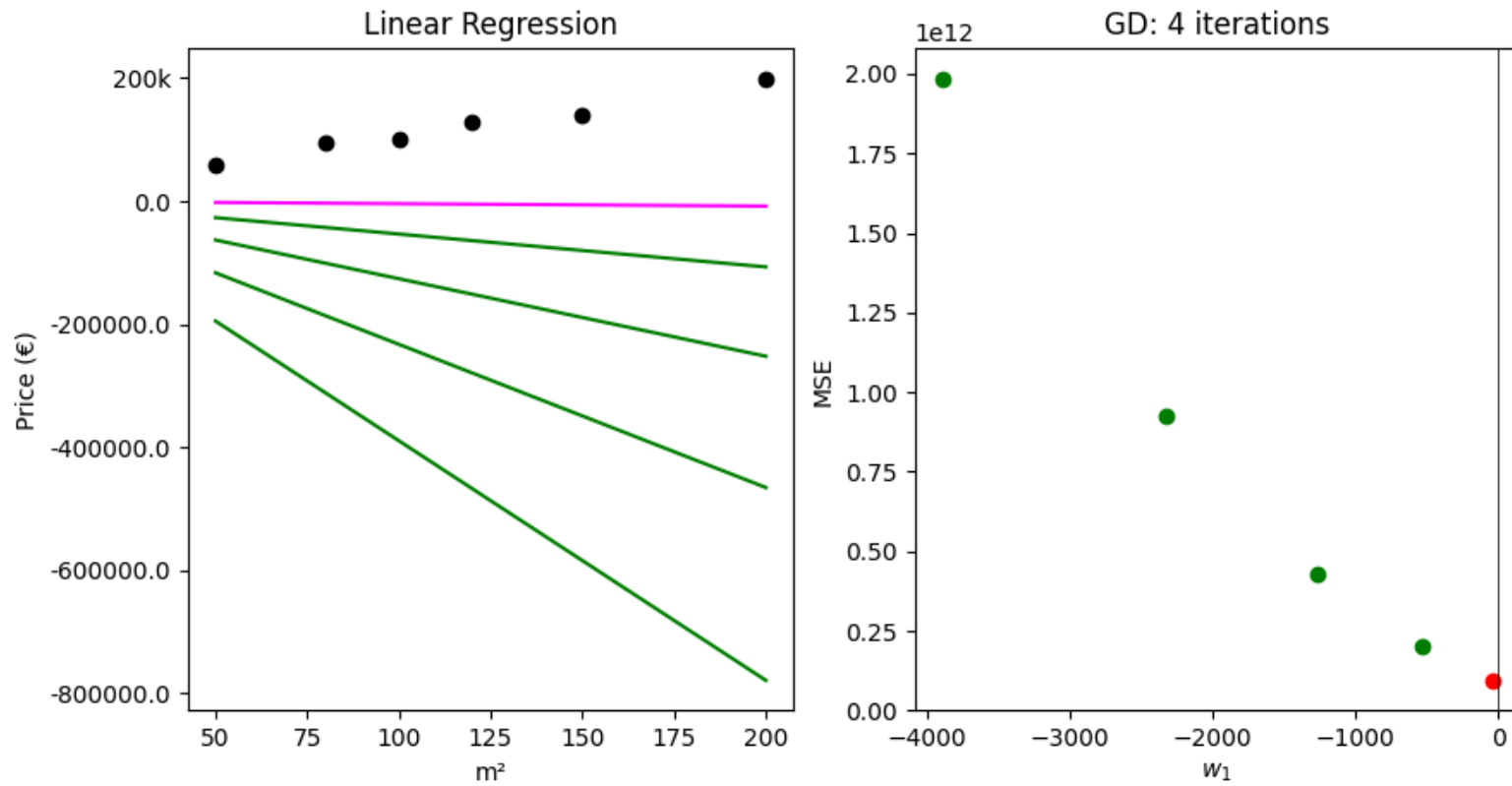
Slope: -49,419,201



```
In [41]: w1 = plot_lr_gd(w1, x, y, iters=4)
```

New w_1 : -40.0

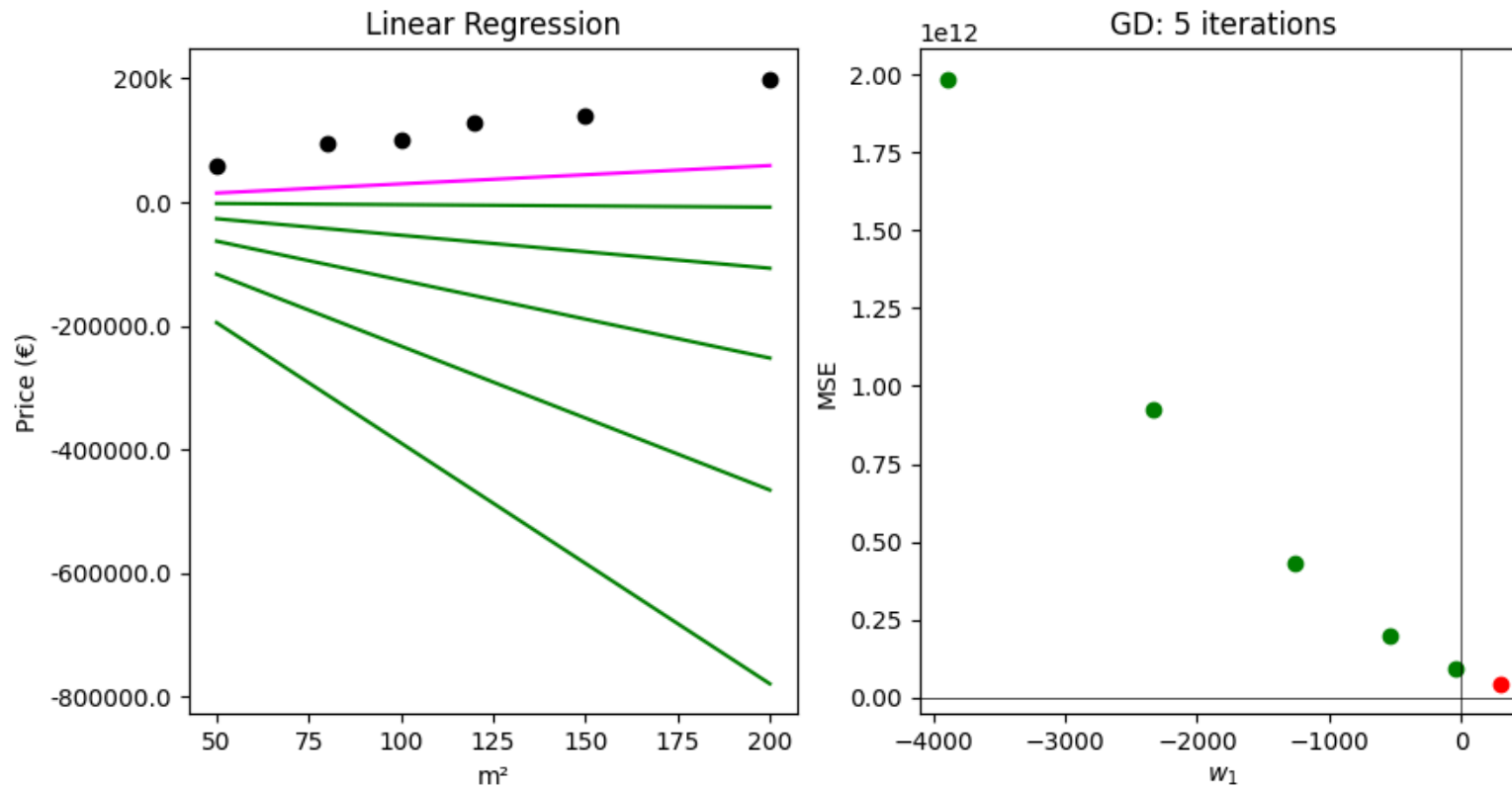
Slope: -33,638,003




```
In [42]: w1 = plot_lr_gd(w1, x, y, iters=5)
```

New w_1 : 296.4

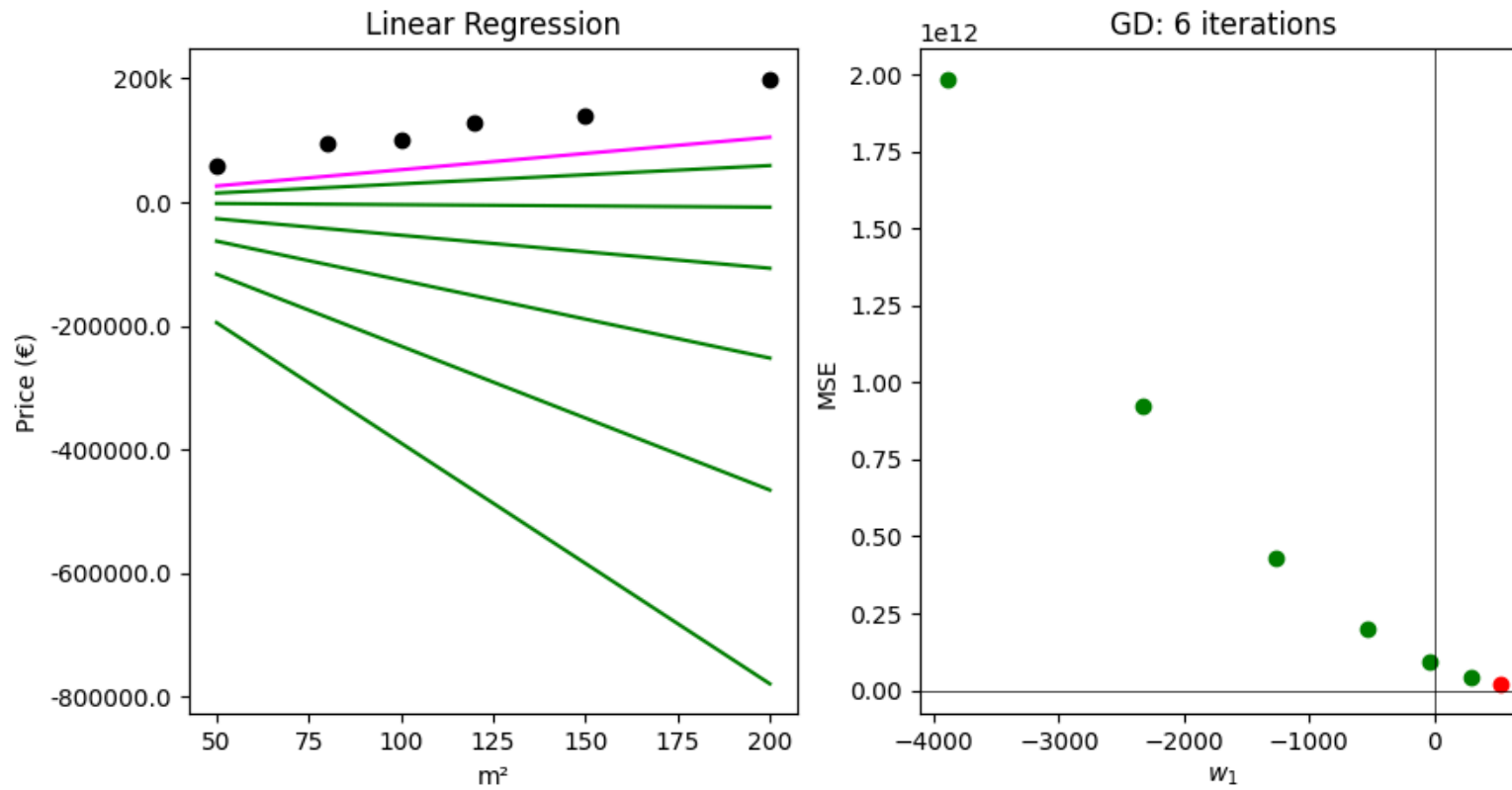
Slope: -22,896,267



```
In [43]: w1 = plot_lr_gd(w1, x, y, iters=6)
```

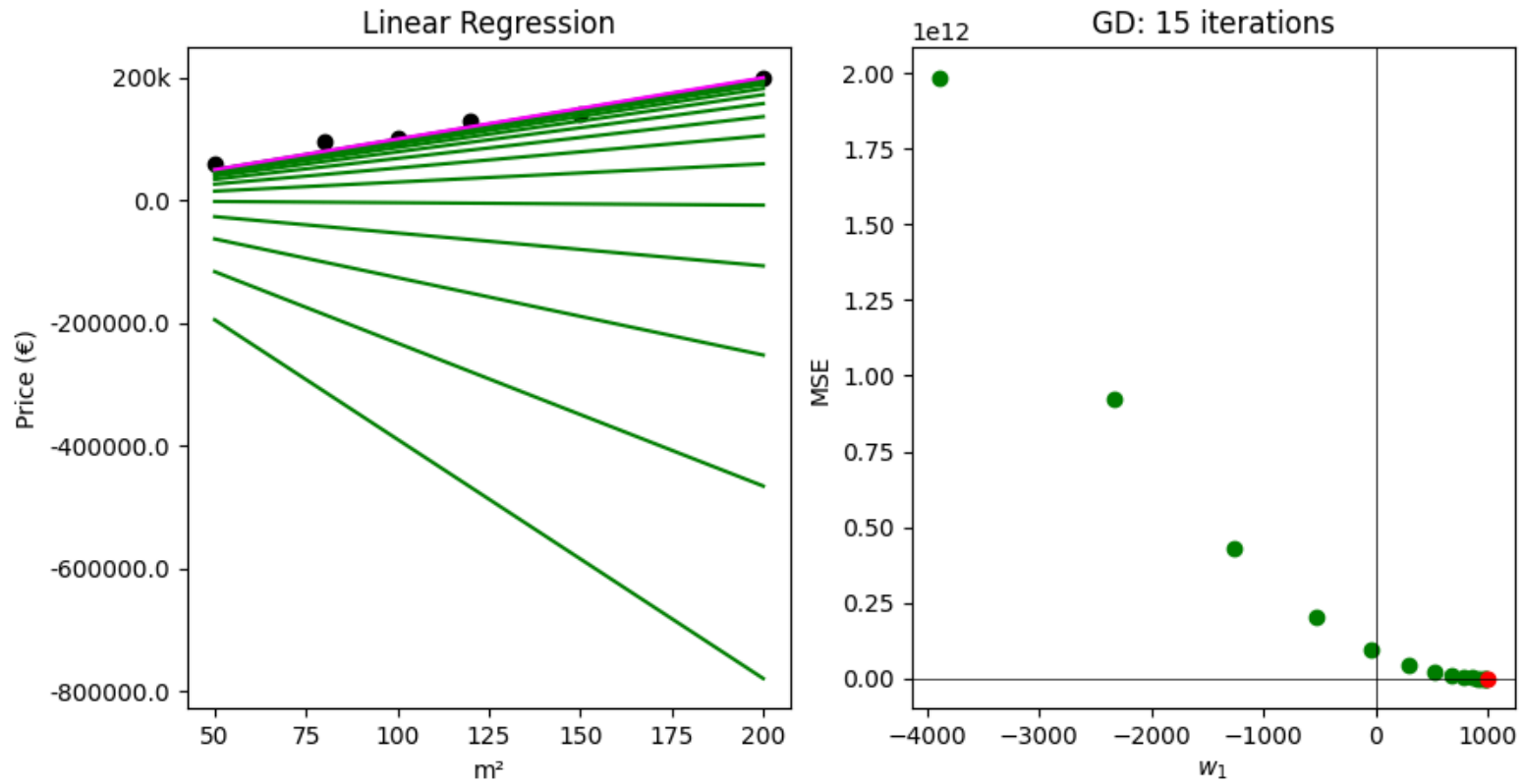
New w1: 525.3

Slope: -15,584,726



```
In [44]: w1 = plot_lr_gd(w1, x, y, iters=15)
```

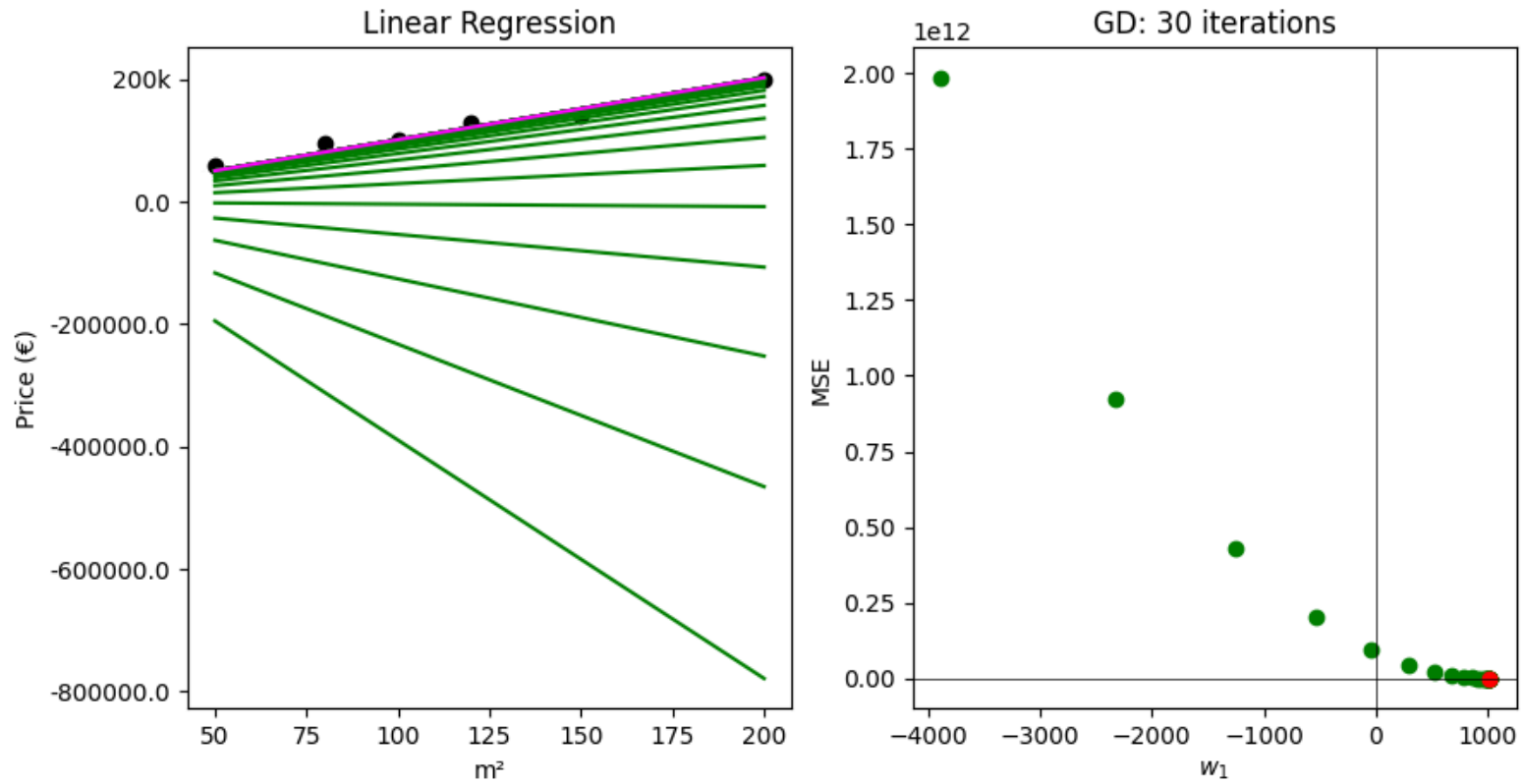
New w1: 998.1
Slope: -488,775



```
In [45]: w1 = plot_lr_gd(w1, x, y, iters=30)
```

New w1: 1013.3

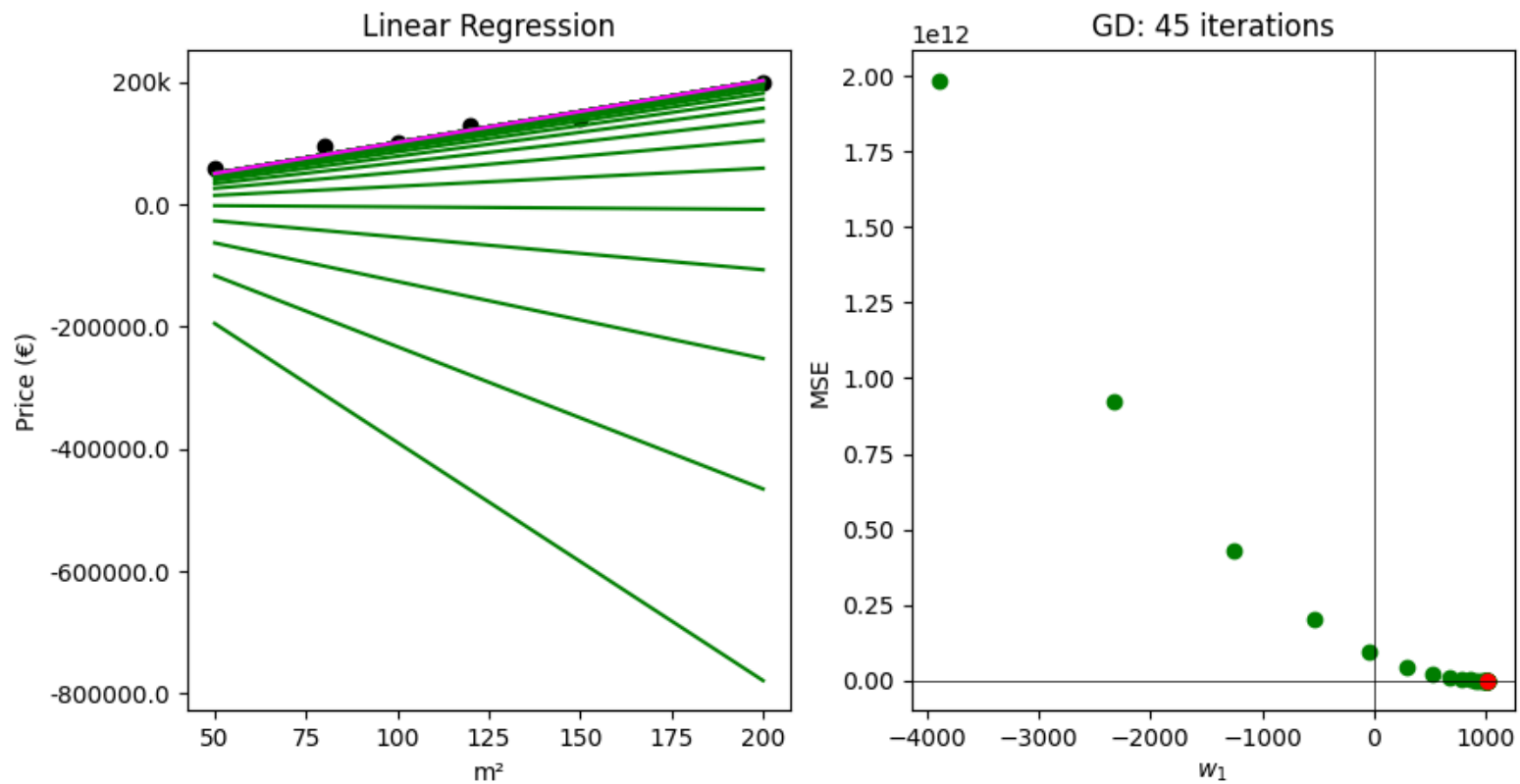
Slope: -1,524



```
In [46]: w1 = plot_lr_gd(w1, x, y, iters=45)
```

New w1: 1013.4

Slope: -4



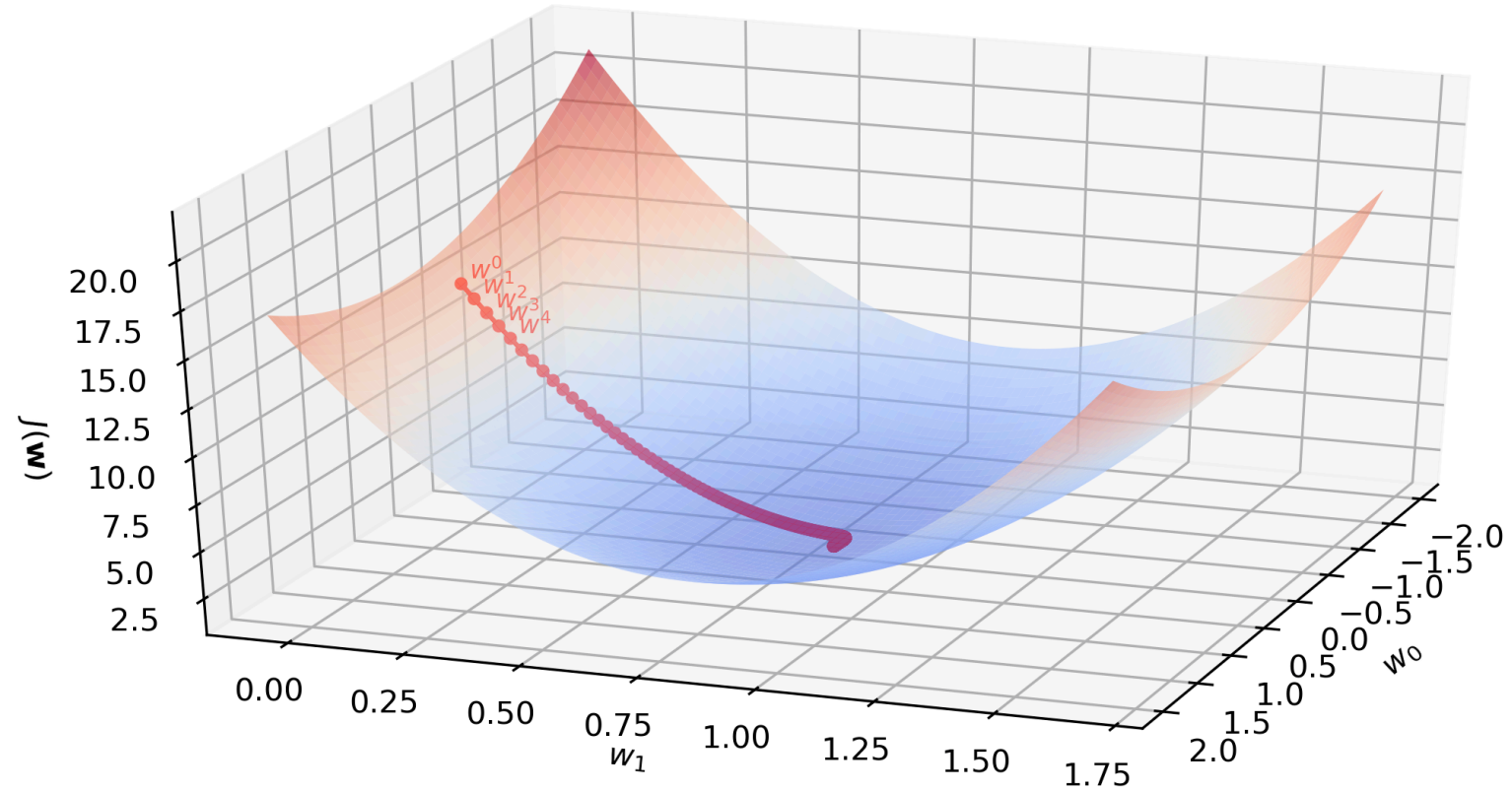
Optimitzar w_0 i w_1

- Abans, per simplicitat, hem suposat $w_0 = 0$ i hem optimitzat w_1 .
- Si volem optimitzar tota l'equació de la recta,
 - w_0 i w_1 ,
- El gradient descent funciona igual però en 3D, amb els eixos: w_0 , w_1 i MSE .

```
In [48]: Image('assets/GD_w0_w1.png', width=800)
```

Out[48]:

Path of the converging gradient descent with $\alpha=0.001$



Gradient Descent per a w_N

- Inicialitza pesos "aleatoris" w_N : un per cada feature més w_0 .
- Repeteix fins a la convergència o fins a arribar al màxim d'iteracions:
 - Fes les prediccions.
 - $\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$ (en el cas de la LR)
 - Calcula la loss (p. ex., MSE o Logloss).
 - $MSE = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$
 - Calcula el gradient (és a dir, la derivada de la loss respecte a cada w_i).
 - Actualitza w_i :
 - $w_i = w_i - \alpha \cdot \nabla_{w_i} MSE$

Forma vectoritzada

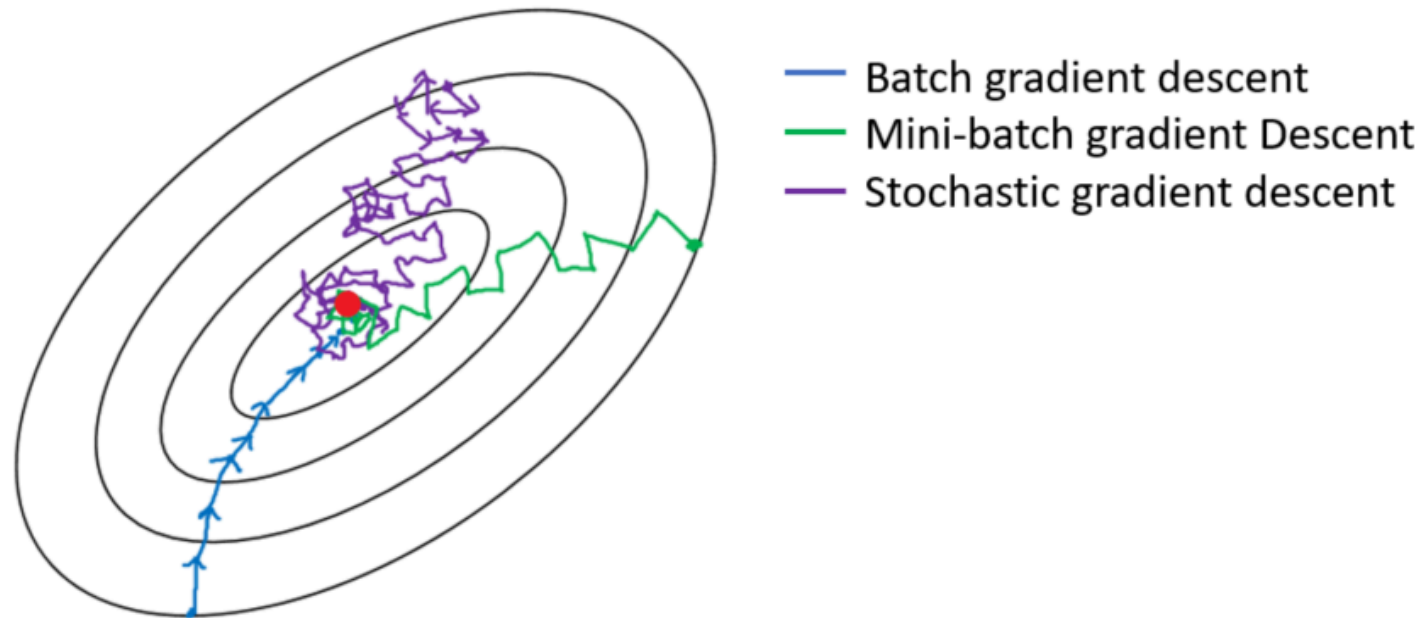
- Computacionalment més eficient.
- És el mateix que abans però fent servir àlgebra lineal.
- Donada una matriu de features X .

- Inicialitza un vector de pesos "aleatori" $\mathbf{w}$.
- Repeteix fins a la convergència o fins a arribar al màxim d'iteracions:
 - Calcula el vector de prediccions $\hat{\mathbf{y}}$.
 - $\hat{\mathbf{y}} = w_0 + \mathbf{X}\mathbf{w}$ (en el cas de la LR)
 - Calcula la loss (p. ex., MSE o Logloss).
 - $MSE = \frac{1}{N} \sum (\hat{y}_i - y_i)^2$
 - Calcula el gradient $\nabla_{\mathbf{w}} MSE$.
 - Actualitza $\mathbf{w}$:
 - $\mathbf{w} = \mathbf{w} - \alpha \cdot \nabla_{\mathbf{w}} MSE$

Tipus de GD

```
In [49]: Image('assets/types_gd.png', width=800)
```

Out[49]:



Batch GD

- Fins a la convergència o el màxim d'iteracions:
 - Calcula l'error per a cada mostra del train set.
 - Actualitza els pesos després d'avaluar totes les mostres d'entrenament.
- Avantatges:
 - Convergència estable.
 - Gradient consistent.
- Inconvenients:
 - Costós computacionalment.
 - Tendència a quedar atrapat en mínims locals.

Stochastic GD

- Fins a la convergència o el màxim d'iteracions:
 - Per a cada mostra del train set:
 - Calcula l'error per a una sola mostra.
 - Actualitza els pesos a partir d'aquesta mostra.
- Avantatges:
 - Actualitzacions ràpides.
 - Possibilitat d'escapar de mínims locals.
- Inconvenients:
 - Convergència sorollosa.
 - Gradient inconsistent.

Mini-batch GD

- Divideix el train set en lots petits (mini-batches).
- Fins a la convergència o el màxim d'iteracions:
 - Aplica el batch GD a cada mini-batch.
- Avantatges i inconvenients mixtos del batch i del stochastic GD.

```
In [50]: Image('assets/types_gd.png', width=800)
```

Out[50]:

